PDF Report Requirements

1. Summary of the Source

1.1 Background

Network Intrusion Detection Systems (IDSs) are an essential component of modern cybersecurity infrastructures. Their objective is to monitor network traffic and automatically distinguish between legitimate communication and malicious activities. As the volume of network traffic continues to grow, manually inspecting every connection becomes impractical. Consequently, machine learning has become an attractive solution because it can automatically learn patterns that differentiate normal traffic from attacks.

The source selected for this project is a public GitHub implementation entitled "NSL-KDD Network Intrusion Detection." The repository demonstrates how classical machine learning algorithms can be applied to the NSL-KDD benchmark dataset in order to classify network connections as either normal or malicious. The project compares several supervised learning algorithms and concludes that ensemble-based methods, particularly Random Forest, provide strong classification performance.

1.2 Problem Statement

The primary problem addressed by the selected project is binary network intrusion detection. Given a network connection described by a collection of traffic statistics and protocol-related features, the goal is to determine whether the connection represents legitimate network activity or an attack.

This problem is important because successful cyberattacks may compromise confidential information, disrupt services, or provide unauthorized access to computer systems. An effective intrusion detection system can identify suspicious behavior early enough to allow security analysts to respond before significant damage occurs.

However, intrusion detection is a challenging machine learning problem. Attack behaviors constantly evolve, some attack categories are extremely rare, and legitimate network traffic often exhibits high variability. These characteristics make it difficult to design models that achieve both high attack detection rates and low false alarm rates.

1.3 Proposed Solution

The selected source proposes solving the intrusion detection problem using supervised machine learning. Instead of manually defining detection rules, the algorithms learn the

relationship between network traffic characteristics and attack labels from historical examples.

The original project compares several commonly used classifiers, including Decision Trees, Random Forests, Support Vector Classification (SVC), and K-Nearest Neighbors (KNN). Their performance is primarily evaluated using classification accuracy on the NSL-KDD benchmark dataset. According to the original repository, ensemble learning techniques provide the strongest predictive performance because they combine multiple decision trees and therefore reduce the variance associated with individual trees.

The implementation includes the complete training pipeline, allowing users to reproduce the reported experiments and compare multiple classification models under identical experimental conditions.

1.4 Dataset

The experiments are performed using the NSL-KDD dataset, which is an improved version of the well-known KDD Cup 1999 intrusion detection dataset. The original KDD'99 benchmark contained a very large number of duplicated records, causing many machine learning algorithms to achieve unrealistically high performance by repeatedly observing identical examples.

NSL-KDD was introduced to address this limitation by removing many duplicated observations and creating a more balanced benchmark. Each network connection is represented by 41 descriptive features together with an attack label and a difficulty score. The features describe multiple aspects of network behavior, including protocol information, service type, connection duration, traffic statistics, content-based characteristics, and destination-host statistics.

One important characteristic of NSL-KDD is that it provides predefined training and testing sets. The official testing set intentionally contains more difficult examples and attack distributions that differ from those observed during training. Consequently, evaluating models on the official test set provides a more realistic estimate of generalization than randomly splitting a single dataset.

Although NSL-KDD remains one of the most widely used educational benchmarks for intrusion detection, it is important to recognize that it represents historical network traffic collected many years ago. Therefore, its ability to reflect current enterprise network environments is limited.

1.5 Methodology

The methodology followed in the original project consists of four main stages.

First, the NSL-KDD dataset is loaded and preprocessed. Numerical and categorical attributes are prepared for machine learning, while attack labels are converted into a binary classification problem distinguishing normal traffic from malicious traffic.

Second, multiple supervised learning algorithms are trained using the training portion of the dataset. The original implementation focuses primarily on comparing Decision Trees, Random Forests, Support Vector Classification, and K-Nearest Neighbors.

Third, the trained models are evaluated on the official testing dataset. The original project mainly emphasizes overall classification accuracy when comparing the different algorithms.

Finally, the best-performing classifier is identified according to the reported evaluation results, leading the authors to conclude that ensemble learning methods provide the strongest overall performance for the NSL-KDD benchmark.

The notebook developed in this project reproduces the main ideas of the original implementation while extending the evaluation considerably. In addition to reproducing the supervised learning pipeline, the notebook includes comprehensive exploratory data analysis, feature engineering, leakage-safe preprocessing pipelines, cross-validation, multiple evaluation metrics, threshold analysis, permutation feature importance, attack-family error analysis, and an additional unsupervised Isolation Forest experiment. These extensions make it possible to evaluate not only whether the original results can be reproduced, but also whether the conclusions drawn by the original project remain valid under a more rigorous experimental methodology.

2. Critical Evaluation of the Selected Source

2.1 Overview

The selected GitHub project provides a clear and accessible implementation of classical machine learning techniques for network intrusion detection using the NSL-KDD dataset. The repository is well organized and demonstrates how multiple supervised learning algorithms can be trained and evaluated on a widely recognized cybersecurity benchmark. From an educational perspective, the project successfully introduces the complete workflow of a supervised intrusion detection system and provides a useful starting point for understanding the application of machine learning in cybersecurity.

Although the implementation achieves its primary objective, reproducing the experiments revealed several methodological limitations. Most of these limitations do not originate from coding mistakes but rather from the evaluation methodology and the assumptions made when interpreting the results. For this reason, the present project extends the original implementation by performing a more comprehensive experimental analysis.

2.2 Strengths of the Original Project

One of the main strengths of the selected project is its simplicity. The repository is relatively easy to understand, allowing students and researchers to reproduce the experiments without requiring specialized hardware or complicated software dependencies. This accessibility increases the educational value of the project and makes it appropriate for demonstrating supervised intrusion detection techniques.

Another important advantage is the use of the NSL-KDD benchmark. Despite its age, NSL-KDD remains one of the most frequently used benchmark datasets in cybersecurity research. Compared with the original KDD Cup 1999 dataset, NSL-KDD removes many duplicated records that previously caused unrealistically optimistic classification results. Consequently, it provides a more challenging evaluation environment while still allowing direct comparison with numerous published studies.

The original project also compares multiple machine learning algorithms instead of evaluating only a single classifier. Comparing several models allows readers to understand how different learning approaches behave under identical experimental conditions. In particular, the inclusion of both linear and nonlinear classifiers demonstrates the advantages of ensemble methods for complex cybersecurity datasets.

Finally, the repository is publicly available, which supports reproducible research. The code, dataset references, and experimental procedure can all be accessed by other researchers, making it possible to verify the reported results independently.

2.3 Weaknesses of the Original Project

Although the project successfully demonstrates supervised intrusion detection, several methodological weaknesses became apparent during reproduction.

The most significant limitation is the heavy reliance on overall classification accuracy. Accuracy measures the proportion of correctly classified observations, but it does not distinguish between false positives and false negatives. In cybersecurity applications, these two error types have very different operational consequences. A false negative allows an attack to remain undetected, whereas a false positive primarily increases analyst workload. Consequently, evaluating an intrusion detection system solely through accuracy may lead to misleading conclusions regarding its practical usefulness.

A second limitation is the absence of detailed error analysis. The original implementation evaluates binary classification performance but does not investigate which attack categories are correctly detected and which remain difficult. The experiments performed in this project demonstrate that the detection performance differs substantially across attack families. Denial-of-Service (DoS) and Probe attacks achieve relatively high detection rates, whereas Remote-to-Local (R2L) and User-to-Root (U2R) attacks remain considerably more challenging. This important observation cannot be identified through overall accuracy alone.

The original project also provides only limited discussion of data preprocessing decisions. Categorical encoding, feature engineering, skewed numerical distributions, feature redundancy, and preprocessing pipelines all influence model performance. Explicitly documenting these decisions improves both interpretability and reproducibility.

Another weakness is the absence of threshold analysis. Most machine learning classifiers output probabilities, yet the original project implicitly assumes that the default classification threshold of 0.50 is always appropriate. In practice, cybersecurity environments often require different operating points depending on the acceptable balance between false alarms and missed attacks. The threshold analysis performed in this project demonstrates that modifying the classification threshold changes the trade-off between precision and recall considerably.

2.4 Comparison with the Reproduced Experiments

The experiments conducted in this notebook generally support the main conclusion of the original project: classical machine learning algorithms are capable of learning meaningful patterns within the NSL-KDD dataset and can successfully distinguish many malicious connections from legitimate traffic.

However, the reproduced experiments also provide several additional insights that significantly refine the original conclusions.

First, cross-validation produced extremely high performance for all supervised models, while evaluation on the official NSL-KDD test set resulted in noticeably lower scores. This difference demonstrates that evaluation on unseen data with a different distribution is considerably more challenging than ordinary cross-validation. Consequently, cross-validation alone should not be interpreted as evidence of real-world performance.

Second, evaluating multiple performance metrics revealed a more complete picture of model behavior. The Decision Tree achieved the strongest balance between precision and recall according to the F2-score, whereas Random Forest produced competitive accuracy but slightly lower recall. These observations suggest that selecting the "best" model depends on the evaluation objective rather than on accuracy alone.

Third, the attack-family analysis revealed substantial differences in detection performance across attack categories. Common attacks such as DoS were detected with relatively high success, while rare attacks including R2L and U2R remained significantly more difficult to classify correctly. These findings indicate that aggregate evaluation metrics conceal important operational weaknesses.

Finally, the additional unsupervised Isolation Forest experiment demonstrated that anomaly detection techniques can identify many malicious connections even without attack labels during training. Although supervised learning achieved stronger overall performance, the anomaly detection experiment illustrates an alternative approach that may complement supervised intrusion detection systems in practice.

2.5 Practical Implications

The results obtained in this project indicate that the selected GitHub implementation is valuable as an educational benchmark rather than a production-ready intrusion detection system.

The experiments clearly demonstrate that machine learning can successfully model historical network traffic represented by NSL-KDD. Nevertheless, the dataset itself has important limitations. It represents historical network behavior, lacks temporal information, and does

not capture modern cloud infrastructures, encrypted communication, Internet of Things devices, or current attacker techniques. Therefore, high performance on NSL-KDD should not automatically be interpreted as evidence that the same models will achieve comparable performance in contemporary operational environments.

Future intrusion detection systems should therefore be evaluated using more recent datasets, temporal validation strategies, probability calibration, attack-specific evaluation metrics, and continuous monitoring for concept drift.

2.6 Overall Assessment

Overall, the selected project provides an effective demonstration of classical supervised machine learning for intrusion detection and successfully illustrates the potential of ensemble learning techniques on the NSL-KDD benchmark. The implementation is reproducible, educational, and technically sound.

Nevertheless, the present reproduction demonstrates that a more rigorous evaluation methodology leads to more balanced conclusions. Considering additional performance metrics, threshold analysis, feature importance, attack-family analysis, and unsupervised anomaly detection reveals important characteristics of model behavior that are not apparent from overall accuracy alone.

For these reasons, the main conclusions of the original project are **partially supported**. The experimental evidence confirms that machine learning is capable of detecting network intrusions within the NSL-KDD benchmark, but it also demonstrates that evaluating such systems requires considerably more comprehensive analysis before conclusions regarding practical deployment can be justified.

3. Feature Engineering Analysis

3.1 Introduction

Feature engineering is one of the most important stages in any machine learning project. The quality of the input features directly influences the model's ability to identify meaningful patterns and distinguish malicious network traffic from legitimate communication.

The original GitHub project focuses mainly on model training and provides only limited discussion of preprocessing decisions. Therefore, one of the objectives of this reproduction was to design a more systematic preprocessing pipeline while preserving the original learning task. The feature engineering process was guided by both machine learning principles and cybersecurity domain knowledge

3.2 Data Cleaning

The first preprocessing stage consisted of examining the quality of the dataset.

The exploratory analysis showed that the NSL-KDD dataset contains no missing values in either the official training or testing sets. Consequently, no observations needed to be removed because of incomplete information. Nevertheless, median and most-frequent imputers were included inside the machine learning pipelines. This design choice ensures that the preprocessing pipeline remains robust if future datasets contain missing values and guarantees consistent preprocessing during cross-validation.

The analysis also identified one constant feature, **num_outbound_cmds**, which contained the same value for every observation. Because a constant feature cannot contribute to classification, it was removed before model training. Eliminating such variables slightly reduces computational complexity while avoiding unnecessary inputs to the learning algorithms.

Finally, the **difficulty** column was excluded from model training. Although this variable is included in the NSL-KDD dataset, it represents metadata created during dataset construction rather than information that would be available in a real intrusion detection system. Including this attribute could therefore introduce unrealistic information into the prediction process

3.3 Target Construction

The original dataset contains numerous individual attack names, including DoS, Probe, Remote-to-Local (R2L), and User-to-Root (U2R) attacks. For the primary classification task, these labels were converted into a binary target variable:

- **0** – Normal network traffic
- **1** – Malicious network traffic

This transformation follows the objective of the selected GitHub project and allows direct comparison with the reproduced experiments.

Although binary labels were used during model training, the original attack names were preserved separately. This made it possible to perform detailed attack-family error analysis after model evaluation without affecting the supervised learning process.

3.4 Categorical Feature Encoding

Three variables in NSL-KDD are categorical:

- protocol_type
- service
- flag

These variables do not possess a natural numerical ordering. Assigning arbitrary integers to these categories could incorrectly suggest ordinal relationships that do not exist.

To avoid this problem, One-Hot Encoding was applied to all categorical variables. Each category was converted into a separate binary indicator variable, allowing the learning algorithms to treat each protocol, service, and connection state independently.

The encoder was configured with **handle_unknown="ignore"**, ensuring that categories appearing only in the official testing dataset would not cause prediction failures. This decision improves robustness and reflects good machine learning practice.

3.5 Numerical Feature Transformation

Many numerical variables in the NSL-KDD dataset exhibit highly skewed distributions. Variables such as connection duration, transmitted bytes, and connection counts contain many small values together with a relatively small number of extremely large observations.

Instead of removing these observations, a logarithmic transformation using log(1+x) was applied to selected non-negative numerical variables. This transformation compresses extremely large values while preserving the relative ordering of observations. As a result, numerical distributions become more stable and easier for linear learning algorithms to model.

For Logistic Regression, numerical features were additionally standardized using StandardScaler. Standardization ensures that variables measured on different scales contribute more equally during optimization and prevents features with large numerical ranges from dominating the learning process.

Tree-based algorithms such as Decision Trees and Random Forests do not require standardized inputs because they partition the feature space using threshold comparisons. Consequently, these models used a separate preprocessing pipeline without feature scaling.

3.6 Engineered Features

In addition to the original dataset variables, several new features were constructed to summarize important aspects of network behavior.

Total Bytes

The first engineered feature combines the number of transmitted and received bytes into a single measure representing the total communication volume. This feature provides a more complete description of connection size than either variable individually.

Source-to-Destination Byte Ratio

The ratio between transmitted and received bytes captures communication asymmetry. Certain attacks produce unusually large differences between incoming and outgoing traffic, making this feature potentially informative for intrusion detection.

Total Error Rate

Multiple network error indicators were combined into a single aggregated measure representing the overall connection error behavior. This feature summarizes abnormal communication patterns while reducing fragmentation across multiple correlated variables.

Total Host Error Rate

Destination-host error statistics were similarly combined to capture repeated abnormal behavior associated with particular hosts.

Connection Intensity

The connection intensity feature combines connection count statistics into a single indicator describing the overall communication activity observed during a monitoring window. High connection intensity may indicate scanning behavior or denial-of-service attacks.

These engineered variables were not intended to replace the original attributes but rather to provide higher-level summaries that may simplify the learning task.

3.7 Correlation and Redundancy Analysis

Before model training, Spearman correlation analysis was performed on numerical variables.

Spearman correlation was selected because many numerical features are strongly skewed and several variables represent bounded rates rather than normally distributed continuous measurements. Unlike Pearson correlation, Spearman correlation evaluates monotonic relationships using ranked observations and is therefore less sensitive to non-normality and extreme values.

The analysis identified several highly correlated feature pairs, indicating that certain traffic statistics describe similar aspects of network behavior. However, these features were intentionally retained. Removing correlated variables may improve interpretability for linear models, but tree-based ensemble methods frequently benefit from correlated predictors because they may capture complementary decision boundaries.

Consequently, redundancy was documented rather than automatically eliminated.

3.8 Effect on Model Performance

The preprocessing pipeline contributed to both robustness and interpretability.

Removing constant features reduced unnecessary complexity, while categorical encoding allowed the algorithms to process protocol information correctly. Logarithmic transformation reduced the influence of extreme numerical values, particularly for Logistic Regression, and the engineered aggregate features introduced higher-level representations of network traffic behavior.

Although the contribution of individual preprocessing decisions cannot be isolated perfectly, the final experiments demonstrated that the resulting feature set allowed all supervised models to achieve strong classification performance on the official NSL-KDD benchmark.

Furthermore, the permutation importance analysis showed that both original and engineered features contributed meaningfully to prediction quality. Features describing traffic volume, communication services, and destination-host behavior consistently appeared among the most influential predictors, suggesting that the feature engineering process successfully captured characteristics associated with malicious network activity.

3.9 Summary

The feature engineering strategy adopted in this project combined careful preprocessing with domain-specific feature construction. Instead of relying exclusively on the original dataset representation, the preprocessing pipeline incorporated data cleaning, categorical encoding, numerical transformations, engineered aggregate variables, and leakage-safe preprocessing pipelines.

These modifications improved the interpretability and robustness of the machine learning workflow while maintaining compatibility with the original GitHub implementation. The final experimental results indicate that thoughtful feature engineering plays an essential role in developing reliable intrusion detection systems and should be considered alongside model selection when designing practical cybersecurity solutions.

4. Reproducibility Analysis

4.1 Introduction

Reproducibility is a fundamental principle of scientific research and machine learning. A project is considered reproducible if another researcher can obtain comparable results by following the published methodology, using the same dataset, and executing the provided implementation. In cybersecurity research, reproducibility is particularly important because it allows researchers to verify experimental claims, compare different approaches fairly, and build upon previous work.

One objective of this project was therefore to evaluate not only the predictive performance of the selected GitHub repository, but also the ease with which its experiments could be reproduced and extended

4.2 Reproducibility of the Original Project

The selected GitHub repository provides publicly available source code together with references to the NSL-KDD dataset. The implementation is relatively compact and easy to understand, making it suitable for educational purposes.

Most of the main experimental steps can be reproduced without substantial modifications. The repository clearly demonstrates how the dataset is loaded, how machine learning models are trained, and how predictions are generated.

However, several reproducibility challenges became apparent during the reproduction process.

First, the original repository does not specify exact package versions. Machine learning libraries such as scikit-learn continue to evolve, and newer versions occasionally modify function names, parameters, or default behavior. During reproduction, several small compatibility changes were required to ensure that the notebook executed correctly under the current Google Colab environment.

Second, the repository assumes that the dataset is already available. Although the NSL-KDD dataset remains publicly accessible through several mirrors, download locations may change over time. To improve robustness, the notebook developed in this project automatically downloads the required dataset files whenever possible and provides clear fallback instructions if automatic download fails.

Third, the preprocessing methodology is only briefly described in the original implementation. Several important design choices—including categorical encoding, feature scaling, leakage prevention, and preprocessing pipelines—are not discussed in detail. Without

explicit documentation, reproducing identical preprocessing decisions becomes more difficult.

4.3 Improvements Introduced in This Project

Several modifications were introduced to improve the reproducibility of the experimental workflow.

The notebook follows a clearly structured sequence beginning with data loading and exploratory analysis before progressing through preprocessing, feature engineering, model training, evaluation, and conclusions. Each stage is documented using explanatory Markdown cells that describe both the implementation and the reasoning behind each decision.

The preprocessing workflow is implemented using scikit-learn pipelines. This approach guarantees that all preprocessing operations—including imputation, categorical encoding, logarithmic transformations, and feature scaling—are learned exclusively from the training data during cross-validation. Using pipelines also minimizes the risk of accidental data leakage and makes the implementation easier to reproduce.

Randomness is controlled consistently throughout the notebook by using a fixed random seed. As a result, repeated executions produce highly similar outputs, reducing unnecessary variability between runs.

In addition, the notebook automatically saves the principal experimental outputs, including evaluation tables, feature importance rankings, threshold analysis, and attack-family results. These exported files facilitate further analysis without requiring the complete notebook to be rerun.

4.4 Remaining Limitations

Although the notebook substantially improves reproducibility, several limitations remain.

The experiments rely on the NSL-KDD benchmark, which represents historical network traffic collected many years ago. Because the original network environment cannot be recreated, reproducing the dataset generation process itself is impossible.

Furthermore, the dataset does not contain timestamps describing when individual network connections occurred. Consequently, temporal validation, concept-drift analysis, and chronological evaluation cannot be reproduced.

Machine learning algorithms also depend on implementation details provided by external software libraries. Minor numerical differences may appear when executing the notebook on different operating systems, hardware platforms, or future library versions. These differences are expected and generally do not affect the overall conclusions of the experiments.

Finally, although the notebook reproduces the experimental methodology, it cannot verify whether the reported benchmark represents the behavior of modern production networks. Real enterprise environments contain different protocols, encrypted communication, cloud infrastructures, and continually evolving attack techniques.

4.5 Overall Assessment

Overall, the selected GitHub repository can be considered reasonably reproducible. The publicly available implementation allows researchers to repeat the principal experiments and obtain comparable results using the NSL-KDD benchmark.

Nevertheless, the reproduction process demonstrated that reproducibility extends beyond simply executing existing code. Careful documentation of preprocessing decisions, dependency management, evaluation methodology, and experimental assumptions is equally important.

The notebook developed in this project addresses many of these issues by providing a fully documented workflow, leakage-safe preprocessing pipelines, automatic dataset handling, explicit evaluation procedures, and detailed explanations accompanying each stage of the analysis. These additions improve transparency and make it easier for future researchers or students to reproduce, understand, and extend the experiments.

5. Experimental Results and Discussion

5.1 Experimental Setup

The experiments were performed using the official NSL-KDD training and testing datasets. Unlike randomly splitting a single dataset, the predefined NSL-KDD train-test separation was preserved throughout the project because it provides a more challenging and realistic evaluation scenario. The official testing set intentionally differs from the training distribution by including more difficult attack instances and different attack frequencies.

Four supervised classifiers were evaluated:

- Logistic Regression
- Decision Tree
- Random Forest
- Dummy Baseline

In addition, an unsupervised Isolation Forest experiment was conducted to investigate whether anomaly detection could identify malicious traffic without using attack labels during training.

To ensure fair comparisons, all preprocessing operations—including imputation, categorical encoding, logarithmic transformations, and feature scaling—were implemented inside leakage-safe scikit-learn pipelines. Cross-validation was performed exclusively on the training data, while the official testing dataset was reserved for the final evaluation.

5.2 Cross-Validation Performance

The first stage of evaluation consisted of five-fold stratified cross-validation on the official training dataset.

All supervised classifiers achieved very high performance during cross-validation. Random Forest obtained the highest average accuracy, closely followed by the Decision Tree model, while Logistic Regression produced slightly lower but still competitive results.

These results indicate that the NSL-KDD training data contains highly informative patterns that can be successfully learned by classical machine learning algorithms. However, cross-validation alone does not measure how well the models generalize to new attack distributions. Since every fold originates from the same training dataset, cross-validation naturally evaluates a distribution that is similar to the data used during learning.

Consequently, these excellent results should be interpreted primarily as evidence that the models successfully fit the training distribution rather than as proof of real-world deployment performance.

5.3 Official Test-Set Performance

The official NSL-KDD testing dataset provides a substantially more demanding evaluation because it contains different attack distributions and more challenging examples.

Compared with the cross-validation results, all supervised models experienced a noticeable decrease in performance. This observation demonstrates that generalization to previously unseen attack patterns remains significantly more difficult than fitting the original training data.

Among the evaluated supervised classifiers, the Decision Tree achieved the strongest overall balance between attack detection and false alarms according to the F2-score. Although Random Forest achieved competitive accuracy and precision, its recall was slightly lower, resulting in a reduced F2-score. Logistic Regression remained highly interpretable and computationally efficient but was less capable of modeling the complex nonlinear relationships present in network traffic.

The Dummy Baseline confirmed that the supervised models learned meaningful predictive patterns rather than simply exploiting the class distribution.

Overall, the experimental results support the original GitHub project's claim that classical machine learning techniques are effective for intrusion detection on the NSL-KDD benchmark. Nevertheless, they also demonstrate that evaluating performance exclusively through accuracy provides only a partial understanding of classifier behavior.

5.4 Evaluation Metrics

Several complementary evaluation metrics were analyzed throughout the experiments.

Accuracy measures the overall proportion of correctly classified observations. Although useful as a general indicator, accuracy alone does not distinguish between false positives and false negatives and may therefore conceal important weaknesses.

Precision evaluates how many predicted attacks actually correspond to malicious traffic. High precision is desirable because it reduces unnecessary security alerts and analyst workload.

Recall measures the proportion of attacks successfully detected by the classifier. In cybersecurity applications, recall is particularly important because missed attacks may lead to system compromise or data loss.

F1-score balances precision and recall equally, while **F2-score** assigns greater importance to recall. Since failing to detect an attack is generally considered more serious than investigating an additional alert, F2-score was selected as the principal metric for comparing the supervised models.

The **Matthews Correlation Coefficient (MCC)** provided an additional balanced measure of classification quality by incorporating all four confusion matrix outcomes simultaneously. Unlike accuracy, MCC remains informative even when class distributions are imbalanced.

Finally, **ROC-AUC** evaluated the ranking ability of each classifier across multiple decision thresholds. Together, these metrics provided a comprehensive assessment of model performance rather than relying on a single numerical indicator.

5.5 Threshold Analysis

One of the extensions introduced in this project was a detailed threshold analysis.

The original implementation implicitly assumed that the default probability threshold of 0.50 represents the optimal operating point. However, the experiments demonstrated that modifying the classification threshold substantially changes the balance between attack recall and false-positive rate.

Lower thresholds increased recall because more suspicious connections were classified as attacks. At the same time, the number of false positives also increased. Conversely, higher thresholds reduced false alarms but caused additional attacks to remain undetected.

This behavior illustrates a fundamental trade-off in intrusion detection systems. The optimal threshold depends on the operational environment rather than on a universal default value. Organizations with high security requirements may intentionally prioritize recall, whereas environments with limited analyst resources may prefer higher precision to reduce alert fatigue.

5.6 Feature Importance Analysis

Permutation importance was used to identify the features that contributed most strongly to prediction performance.

The analysis consistently identified network traffic volume, communication services, and destination-host statistics among the most influential predictors. These findings agree with established cybersecurity intuition because many network attacks generate abnormal communication volumes, unusual service requests, or repeated interactions with specific hosts.

Several engineered features also appeared among the most informative variables, indicating that feature engineering successfully captured meaningful higher-level characteristics of network behavior.

It is important to recognize that feature importance measures predictive contribution rather than causal influence. A highly important feature improves model performance but does not necessarily represent the underlying cause of malicious activity.

5.7 Error Analysis

One of the most informative analyses performed in this project examined classifier performance across individual attack families.

The results revealed substantial differences between attack categories. Denial-of-Service (DoS) and Probe attacks achieved relatively high detection rates, whereas Remote-to-Local (R2L) and User-to-Root (U2R) attacks remained considerably more difficult to identify correctly.

This observation explains why relying solely on overall accuracy may produce overly optimistic conclusions. A classifier may correctly classify the majority of common attacks while still failing to detect several rare but operationally significant attack categories.

The detailed attack-family analysis therefore provides valuable information that cannot be obtained from aggregate evaluation metrics alone.

5.8 Unsupervised Anomaly Detection

The additional Isolation Forest experiment demonstrated that anomaly detection can identify a large proportion of malicious connections without requiring attack labels during training.

Although supervised classifiers achieved stronger overall performance, the anomaly detection experiment illustrates an important alternative approach. In practical cybersecurity environments, obtaining labeled attack data is often expensive and time-consuming. Unsupervised anomaly detection can therefore complement supervised intrusion detection by identifying previously unseen or poorly represented attack types.

Rather than replacing supervised learning, the results suggest that both approaches may provide complementary benefits within a comprehensive intrusion detection framework.

5.9 Discussion

Overall, the experimental results support the principal objective of the selected GitHub project while also extending its conclusions.

The reproduction confirms that classical machine learning algorithms successfully learn meaningful patterns within the NSL-KDD benchmark. However, the additional analyses performed in this project demonstrate that evaluating intrusion detection systems requires considerably more than reporting overall accuracy.

Cross-validation, official test-set evaluation, threshold analysis, attack-family performance, feature importance, and anomaly detection each reveal different aspects of model behavior. Considering these complementary analyses together provides a more realistic understanding of both the strengths and limitations of machine learning for cybersecurity.

The experiments therefore reinforce the importance of comprehensive evaluation methodologies when assessing intrusion detection systems intended for practical deployment.

6. Conclusions and Future Work

6.1 Conclusions

The primary objective of this project was to reproduce, evaluate, and extend a publicly available GitHub implementation for network intrusion detection using the NSL-KDD dataset. Throughout the project, the complete machine learning workflow was examined, including data exploration, preprocessing, feature engineering, supervised model training, model evaluation, and result interpretation.

The reproduction was successful. The implemented notebook reproduced the main ideas of the original project while extending the evaluation methodology considerably. In addition to comparing multiple supervised learning algorithms, the project incorporated leakage-safe preprocessing pipelines, comprehensive exploratory data analysis, feature engineering, cross-validation, threshold analysis, permutation feature importance, attack-family error analysis, and an additional unsupervised anomaly detection experiment.

The experimental results confirmed that classical machine learning algorithms are capable of distinguishing malicious network traffic from legitimate connections on the NSL-KDD benchmark. Decision Tree achieved the best overall balance between precision and recall according to the F2-score, while Random Forest produced competitive performance and Logistic Regression provided an interpretable baseline.

However, the experiments also demonstrated that evaluating intrusion detection systems using overall accuracy alone is insufficient. Although cross-validation produced excellent performance, evaluation on the official NSL-KDD test set showed a noticeable reduction in predictive quality. This finding indicates that generalization to previously unseen attack patterns remains considerably more difficult than fitting the training distribution.

The detailed error analysis further revealed that classifier performance differs substantially across attack families. Common attacks, particularly Denial-of-Service (DoS), were detected with relatively high success, whereas Remote-to-Local (R2L) and User-to-Root (U2R) attacks remained significantly more challenging. This observation emphasizes the importance of evaluating intrusion detection systems using multiple complementary metrics rather than relying on aggregate accuracy alone.

Overall, the results partially support the conclusions of the original GitHub project. The reproduced experiments confirm that classical supervised learning techniques perform well on the NSL-KDD benchmark. Nevertheless, the extended analyses performed in this project demonstrate that a more comprehensive evaluation provides a considerably deeper understanding of model strengths, weaknesses, and practical limitations.

6.2 Lessons Learned

Several important lessons emerged during the reproduction process.

First, data preprocessing proved to be just as important as model selection. Careful handling of categorical variables, logarithmic transformations, engineered features, and leakage-safe preprocessing pipelines contributed to obtaining reliable and reproducible experimental results.

Second, model evaluation should never rely on a single metric. Accuracy alone does not adequately describe classifier performance, particularly in cybersecurity applications where false negatives and false positives have very different operational consequences. Combining Precision, Recall, F1-score, F2-score, Matthews Correlation Coefficient, ROC-AUC, and attack-family analysis provides a much more informative evaluation.

Third, reproducibility requires considerably more than publishing source code. Clear documentation, explicit preprocessing decisions, fixed random seeds, consistent evaluation procedures, and transparent reporting are all essential components of reproducible machine learning research.

Finally, this project demonstrated the value of combining domain knowledge with machine learning techniques. The engineered features were motivated by network behavior rather than purely statistical considerations, illustrating how cybersecurity expertise can improve both interpretability and predictive performance.

6.3 Future Work

Although the reproduced implementation achieved strong performance on the NSL-KDD benchmark, several opportunities remain for future improvement.

A natural extension would be to evaluate the proposed methodology on more recent cybersecurity datasets such as UNSW-NB15, CICIDS2017, or CSE-CIC-IDS2018. These datasets better represent modern network environments, cloud infrastructures, encrypted communication, and contemporary attack techniques.

Future work could also investigate multiclass intrusion detection instead of binary classification. Distinguishing individual attack categories directly would provide security analysts with more detailed information regarding the nature of detected threats.

Another promising direction involves deep learning approaches. Neural-network architectures such as recurrent neural networks, convolutional neural networks, or transformer-based models may capture more complex traffic patterns than classical machine learning algorithms, particularly when large-scale datasets are available.

Probability calibration and automated threshold optimization could further improve operational deployment. Instead of using a fixed decision threshold, future systems could dynamically adjust the operating point according to the current security requirements, expected attack frequency, or analyst workload.

Finally, real-world intrusion detection systems should incorporate continuous monitoring and concept-drift detection. Network traffic evolves over time as new services, applications, and attack techniques emerge. Periodically retraining models and monitoring changes in the underlying data distribution would help maintain predictive performance in operational environments.

6.4 Final Remarks

This project demonstrates that reproducing an existing machine learning implementation provides valuable insight beyond simply executing previously published code. Reproduction encourages careful examination of methodological decisions, evaluation procedures, and experimental assumptions.

By extending the original GitHub project with additional analyses, more rigorous evaluation metrics, and comprehensive feature engineering, this work provides a more complete assessment of classical machine learning for network intrusion detection. The results confirm the educational value of the original implementation while also highlighting the importance of critical evaluation, transparent experimentation, and thoughtful interpretation when applying machine learning techniques to cybersecurity problems.

7. Executive Summary

This project reproduced and extended a publicly available GitHub implementation of a machine learning-based Network Intrusion Detection System (IDS) using the NSL-KDD dataset. The primary objective was to evaluate whether the conclusions presented in the original project could be reproduced while performing a more comprehensive experimental analysis.

The project began with an exploratory analysis of the dataset, including an examination of class distributions, attack families, feature characteristics, missing values, duplicate records, and feature correlations. This analysis provided a better understanding of the data before model development and identified preprocessing decisions that improved the reliability of the experiments.

A complete preprocessing pipeline was then developed. Constant features were removed, categorical variables were encoded using One-Hot Encoding, skewed numerical variables were transformed using logarithmic scaling, and additional engineered features describing network behavior were introduced. All preprocessing operations were implemented inside leakage-safe machine learning pipelines to ensure fair model evaluation.

Four supervised classification models were evaluated: Logistic Regression, Decision Tree, Random Forest, and a Dummy Baseline classifier. In addition, an unsupervised Isolation Forest model was implemented to investigate anomaly detection without attack labels.

The experimental results demonstrated that classical machine learning algorithms successfully detect malicious network traffic within the NSL-KDD benchmark. Cross-validation produced excellent predictive performance; however, evaluation on the official testing dataset showed a noticeable reduction in performance, indicating that generalization to previously unseen attack patterns remains challenging.

Among the evaluated supervised models, the Decision Tree achieved the strongest balance between attack detection and false alarms according to the F2-score, while Random Forest achieved similarly competitive performance. The additional threshold analysis demonstrated that selecting the default classification threshold is not always optimal and that different operating points may better satisfy different security requirements.

A detailed attack-family analysis revealed that common attacks such as Denial-of-Service (DoS) and Probe attacks were detected successfully, whereas Remote-to-Local (R2L) and User-to-Root (U2R) attacks remained significantly more difficult to classify correctly. This finding demonstrates that overall accuracy alone is insufficient for evaluating intrusion detection systems and highlights the importance of using complementary performance metrics.

Overall, the reproduced experiments support the main objective of the selected GitHub project while also demonstrating that more comprehensive evaluation provides a deeper

understanding of classifier performance. The project concludes that feature engineering, careful preprocessing, multiple evaluation metrics, and detailed error analysis are essential components of reliable machine learning systems for cybersecurity.

References

References

Eroglu, M., & Sezgin, M. (n.d.). *NSL-KDD network intrusion detection* [Computer software]. GitHub. https://github.com/Mamcose/NSL-KDD-Network-Intrusion-Detection

Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. *Nature, 585*, 357–362.

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95.

McKinney, W. (2010). Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference* (pp. 56–61).

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Tavallaee, M., Bagheri, E., Lu, W., & Ghorbani, A. A. (2009). A detailed analysis of the KDD CUP 99 data set. In *Proceedings of the 2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications* (pp. 1–6). IEEE.

Wong, J. M. N. (n.d.). *NSL-KDD dataset* [Data set]. GitHub. https://github.com/jmnwong/NSL-KDD-Dataset